

we have an example of a class where the `__init__()` method initializes an instance variable `self.count` (read as “self dot count”) to zero and another method, `increment()`, that simply adds 1 to `self.count`:

```
class MyClass():
    def __init__(self):
        self.count = 0 # create self.count and set it to 0
    def increment(self):
        self.count = self.count + 1 # increment the variable
```

When you instantiate an object from the `MyClass` class, the `__init__()` method runs and sets the value of the instance variable `self.count` to zero. If you then call the `increment()` method, the value of `self.count` goes from zero to one. If you call `increment()` again, the value goes from one to two, and on and on.

Each object created from a class gets its own set of instance variables, independent of any other objects instantiated from that class. In the case of the `LightSwitch` class, there is only one instance variable, `self.switchIsOn`, so every `LightSwitch` object will have its own `self.switchIsOn`. Therefore, you can have multiple `LightSwitch` objects, each with its own independent value of `True` or `False` for its `self.switchIsOn` variable.

Differences Between Functions and Methods

To recap, there are three key differences between a function and a method:

1. All methods of a class must be indented under the `class` statement.
2. All methods have a special first parameter that (by convention) is named `self`.
3. Methods in a class can use instance variables, written in the form `self.<variableName>`.

Now that you know what methods are, I’ll show you how to create an object from a class and how to use the different methods that are available in a class.